

Contents

guide 01 retrieval_augmented_generation	1
0. 这篇论文一句话	2
1. 为什么这篇重要	2
2. 历史语境	2
3. 论文结构地图	3
4. 核心概念	4
4.1 Parametric memory	4
4.2 Non-parametric memory	4
4.3 Retriever	5
4.4 Generator	5
4.5 Latent document	5
5. Figure 1 的系统图	5
6. RAG-Sequence	6
7. RAG-Token	7
8. RAG-Sequence vs RAG-Token 的最小例子	7
9. Training objective	8
10. Decoding	9
11. 实验任务	9
12. Table 1: Open-domain QA	10
13. Table 2 和 Table 3: Generation and FEVER	10
14. Ablations	11
15. 数据形状	11
16. 本地代码映射	12
17. 最小代码片段	13
18. 30 到 60 分钟本地实验	13
19. 证据链总结	14
20. 局限性	15
21. 现代意义	15
22. 常见误解	16
23. 用 AI agent 正确学习这篇	16
24. 闭卷掌握检查	17
25. 最小复述模板	18

guide_01_retrieval_augmented_generation

原论文: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

本地原文 PDF: [learning/rag-essential/paper/01_retrieval_augmented_generation.pdf](#)

作者: Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, Douwe Kiela

机构: Facebook AI Research, University College London, New York University

arXiv: 2005.11401

类型: retrieval-augmented generation, open-domain QA, latent retrieval, seq2seq generation

0. 这篇论文一句话

RAG 把语言模型的参数记忆和一个可检索、可替换、可检查的外部知识库结合起来: 先用 dense retriever 从 Wikipedia index 里取 top K passages, 再让 seq2seq generator 在输入和 retrieved passages 条件下生成答案, 并把 retrieved document 当作 latent variable 做边缘化。

现代很多人说 RAG 时指的是:

```
retrieve chunks  
put chunks into prompt  
ask LLM answer
```

但原始 RAG 论文更具体, 也更数学化:

```
x = input question or claim  
z = retrieved passage as latent document  
y = generated answer or label
```

```
model computes:  
  retriever probability over z  
  generator probability of y given x and z  
  marginal probability of y after summing over z
```

所以这篇不是普通工程教程, 而是“ parametric memory + non-parametric memory” 的生成式概率模型。

1. 为什么这篇重要

在 2020 年前后, 预训练语言模型已经表现出大量事实知识。T5、BART 等模型能在参数里存储知识, 并在下游任务上 fine-tune。但这类纯 parametric model 有三个痛点。

第一, 知识不容易更新。

模型知道什么, 主要写在参数里。如果世界发生变化, 通常需要继续训练或微调。这很慢, 也可能破坏其他能力。

第二, 知识来源不透明。

模型回答一个事实时, 你很难知道它依据了哪段资料。对于知识密集任务, provenance 很重要。

第三, 容易 hallucinate。

如果模型参数中没有足够信息, 或者输入问题需要外部证据, 纯生成模型可能编造看似流畅但错误的答案。

RAG 的动机是:

让模型保留语言生成能力,
同时给它一个外部、可更新、可检查的知识库。

这就是 parametric memory 和 non-parametric memory 的结合。

2. 历史语境

RAG 不是凭空出现。它站在几条线的交汇处。

Open-domain QA:

- 系统需要从大规模语料里找证据并回答问题。
- 早期常见 pipeline 是 retriever + reader。
- retriever 找相关文档，reader 从文档里抽取 answer span。

DPR:

- Dense Passage Retrieval 使用 query encoder 和 document encoder。
- 通过向量内积做 passage retrieval。
- 相比 BM25，dense retriever 能匹配语义而不只依赖词面重合。

REALM 和 ORQA:

- 也把 retriever 做成可微的 latent retrieval。
- 但主要探索 extractive QA 或 masked LM。

BART/T5:

- 预训练 seq2seq 模型成为 NLP 的“workhorse”。
- 能做生成、摘要、问答、分类等任务。

RAG 的贡献是把这些合在一起:

- 用 DPR 做 non-parametric memory access。
- 用 BART 做 parametric seq2seq generator。
- 把 retrieval document 当 latent variable。
- 对 generation 任务做端到端 fine-tuning。
- 不只做 extractive QA，还做 abstractive QA、Jeopardy generation、FEVER classification。

3. 论文结构地图

第 1 节 Introduction:

- 说明纯 parametric LM 的局限。
- 引出 hybrid parametric and non-parametric memory。
- Figure 1 展示整体结构。
- 概述 RAG-Sequence 和 RAG-Token。
- 总结主要实验结果。

第 2 节 Methods:

- 第 2.1 节定义 RAG-Sequence 和 RAG-Token。
- 第 2.2 节介绍 DPR retriever。
- 第 2.3 节介绍 BART generator。
- 第 2.4 节说明 training objective。
- 第 2.5 节说明 decoding。

第 3 节 Experiments:

- Open-domain QA: Natural Questions、TriviaQA、WebQuestions、CuratedTrec。
- Abstractive QA: Open MS-MARCO。
- Jeopardy Question Generation。
- FEVER fact verification。

第 4 节 Results:

- Table 1: open-domain QA test scores。
- Table 2: generation and classification scores。

- Table 3: qualitative examples。
- Table 4: human assessment for Jeopardy generation。
- Table 5: distinct ngram diversity。
- Table 6: retrieval ablations。
- Figure 2: RAG-Token token-level document posterior。
- Figure 3: number of retrieved docs and performance。
- Index hot-swapping experiment。

第 5 节 Related Work:

- Open-domain QA、retrieval-augmented models、retrieve-and-edit。

第 6 节 Discussion:

- 总结 hybrid memory。
- 讨论 retrieval component 的有效性。
- 展示 index 可替换带来的知识更新。
- 指出未来可探索 joint pretraining。

附录:

- 实现细节。
- 数据集大小。
- FEVER 细节。
- null document 机制实验。
- 各任务训练和解码设置。

4. 核心概念

4.1 Parametric memory

Parametric memory 指模型参数里存的知识。

例如 BART 或 T5 在预训练时见过大量文本，于是可能知道一些事实、语法、实体关系和常识。这些知识存在 transformer weights 里。

优点:

- 推理时无需外部检索。
- 生成流畅。
- 能用语言模型能力组合信息。

缺点:

- 更新难。
- 来源不可见。
- 容易过时。
- 可能 hallucinate。

4.2 Non-parametric memory

Non-parametric memory 指外部知识库，在 RAG 中是 Wikipedia passages 的 dense vector index。

优点:

- 可以替换 index 更新知识。
- retrieved passages 可以被检查。

- 可以扩展到很大规模。
- 和模型参数分离。

缺点:

- 检索错误会误导生成。
- 检索增加延迟和成本。
- 文档切分、索引、向量质量都影响效果。
- 只检索到证据不等于生成忠实。

4.3 Retriever

Retriever 根据 input x 找 top K passages z 。

原论文使用 DPR。DPR 是 bi-encoder:

```
q(x) = query encoder output
d(z) = document encoder output
score(x, z) = dot(q(x), d(z))
p_eta(z given x) = softmax over top K scores
```

检索 top K 是 Maximum Inner Product Search, 也就是 MIPS。

4.4 Generator

Generator 根据 input 和 passage 生成 output。

原论文使用 BART-large。输入形式可以理解为:

```
encoder input = x + z
decoder output = y
```

generator probability:

```
p_theta(y_i given x, z, y_before_i)
```

4.5 Latent document

RAG 不把某一个 retrieved passage 当作唯一真相。它把 document z 当作 latent variable。

意思是:

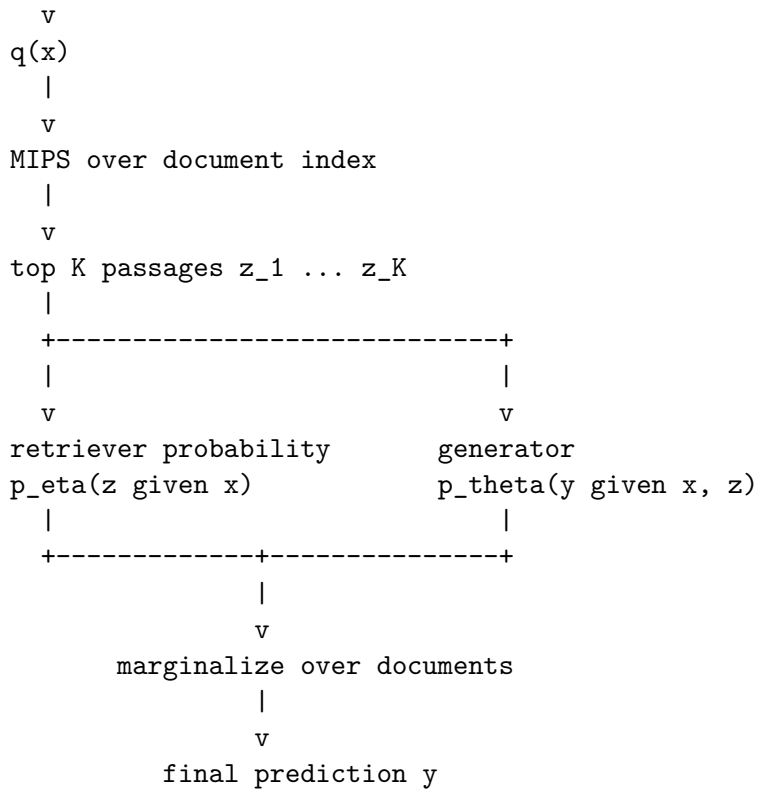
模型不知道到底哪个文档负责最终答案。
所以它对多个可能文档加权求和。

这个边缘化是论文数学的核心。

5. Figure 1 的系统图

Figure 1 展示了完整流程。

```
input x
  |
  v
query encoder
  |
```



注意两个 memory 的位置。

Parametric memory:

- BART generator 的参数。
- 它负责语言能力、生成能力、部分事实补全。

Non-parametric memory:

- Wikipedia dense index。
- 它负责提供外部证据和可更新知识。

6. RAG-Sequence

RAG-Sequence 假设同一个 retrieved document 负责整个 output sequence。

直觉:

先选一个 passage z 。

整段答案 y 都基于这个 passage 生成。

最后对 top K passages 的可能性求和。

公式用导读符号写:

$$\begin{aligned}
 p_{\text{seq}}(y \text{ given } x) = & \\
 & \text{sum over } z \text{ in topK}(x): \\
 & \quad p_{\eta}(z \text{ given } x) \\
 & \quad * \\
 & \quad \text{product over token } i: \\
 & \quad \quad p_{\theta}(y_i \text{ given } x, z, y_{\text{before}_i})
 \end{aligned}$$

适合场景:

- 一个问题主要由一个 passage 支撑。
- 输出较短。
- open-domain QA 中答案通常来自单个证据段。

优点:

- 文档级解释更清楚。
- 每条生成候选可以对应一个主要 passage。

缺点:

- 如果答案需要多个 passages, 单一文档假设会受限。
- decoding 更麻烦, 因为概率不是标准 token-by-token factorization。

7. RAG-Token

RAG-Token 允许每个 token 使用不同 document。

直觉:

生成第一个 token 时, 对 top K passages 加权。
生成第二个 token 时, 再对 top K passages 加权。
每个 token 都可以有不同 document posterior。

公式:

$$p_{\text{tok}}(y \text{ given } x) = \prod_{\text{token } i} \sum_{z \in \text{topK}(x)} p_{\text{eta}}(z \text{ given } x) * p_{\text{theta}}(y_i \text{ given } x, z, y_{\text{before}_i})$$

适合场景:

- 输出需要组合多个证据。
- Jeopardy question generation 这种生成可能引用多个事实。
- 一个文档提供一部分信息, 另一个文档提供另一部分信息。

优点:

- 更灵活。
- 可以在 token level 切换证据来源。
- 可以用标准 autoregressive decoding 的方式近似。

缺点:

- token-level provenance 更复杂。
- 每个 token 的文档 posterior 不一定容易解释给用户。

8. RAG-Sequence vs RAG-Token 的最小例子

假设有两个文档:

d1 probability = 0.7
d2 probability = 0.3

目标答案有两个 token。generator 在每个文档下给出的 token probability:

```
d1: [0.9, 0.8]
d2: [0.1, 0.95]
```

RAG-Sequence:

```
d1 explains whole answer:
  0.7 * 0.9 * 0.8
```

```
d2 explains whole answer:
  0.3 * 0.1 * 0.95
```

```
sum = 0.5325
```

RAG-Token:

```
token 1 marginal:
  0.7 * 0.9 + 0.3 * 0.1 = 0.66
```

```
token 2 marginal:
  0.7 * 0.8 + 0.3 * 0.95 = 0.845
```

```
product = 0.5577
```

这个例子对应本地:

```
learning/rag-essential/src/rag_original_minimal.py
```

它不是为了模拟 BART，而是为了让你把边缘化公式跑成数字。

9. Training objective

训练数据是 input-output pairs:

(x_j, y_j)

模型目标是最大化 target 的 marginal likelihood，等价于最小化 negative marginal log likelihood:

```
loss =
  sum over examples j:
    - log p(y_j given x_j)
```

这里的 $p(y_j \text{ given } x_j)$ 可以来自 RAG-Sequence 或 RAG-Token。

训练时有个重要工程选择:

- query encoder BERT_q 会更新。
- BART generator 会更新。
- document encoder BERT_d 和 index 固定。

为什么固定 document encoder?

如果更新 document encoder，所有 document vectors 都变了，index 需要周期性重建，成本很高。论文没有发现这对 strong performance 必要，所以固定 index。

这带来一个很重要的现代启发:

RAG 的 retriever 可以学习任务相关查询表示，
但知识库 index 本身可以保持稳定。

10. Decoding

RAG-Token 的 decoding 比较自然。

因为它可以写成每个 token 的 transition probability:

```
p_next(y_i) =  
  sum over z:  
    p_eta(z given x) * p_theta(y_i given x, z, y_before_i)
```

所以可以把它放进普通 beam search。

RAG-Sequence 不一样。

它需要对每个 document 做 beam search, 得到候选 answer set。然后对每个候选 answer, 再计算它在不同 documents 下的 generator probability, 并乘上 retriever probability 后求和。

论文把较完整的版本叫 Thorough Decoding。为了效率, 也可以用 Fast Decoding, 近似忽略没有在某一个 document beam 中出现的候选。

这说明:

RAG-Sequence 的假设更文档级,
但 decoding 更麻烦。

RAG-Token 更像标准 autoregressive model,
但解释性更 token-level。

11. 实验任务

Open-domain QA:

- Natural Questions。
- TriviaQA。
- WebQuestions。
- CuratedTrec。
- 指标是 Exact Match。

Abstractive QA:

- Open MS-MARCO。
- 指标包括 Rouge-L 和 Bleu-1。

Jeopardy question generation:

- 给定答案实体, 生成 Jeopardy 风格问题。
- 这要求生成精确事实描述。
- 指标包括 Q-BLEU-1 和 human assessment。

FEVER fact verification:

- 输入 natural language claim。
- 输出 supports、refutes、not enough info。
- RAG 把 class label 当作长度为 1 的 target sequence。
- RAG-Sequence 和 RAG-Token 在这种设置下等价。

12. Table 1: Open-domain QA

Table 1 显示 RAG 在四个 open-domain QA tasks 上表现很强。

重点结论:

- RAG 在 Natural Questions、WebQuestions、CuratedTrec 上达到新的 state of the art。
- TriviaQA 上按 T5-comparable split 也表现强。
- RAG-Sequence 在 Natural Questions 上约 44.5 EM。
- RAG-Token 在 Natural Questions 上约 44.1 EM。
- DPR QA system 使用 retriever、cross-encoder reranker、extractive reader。
- RAG 不需要 extractive reader, 也不需要 reranker, 就能生成答案。

为什么 generation 可能比 extraction 好?

- 有些 documents 提供 clue, 但不直接包含 answer string。
- generator 可以把证据和参数知识结合起来。
- 论文提到在 NQ 中, 即使 correct answer 不在任何 retrieved document 中, RAG 仍有一部分正确率, extractive model 在这种情况下会是 0。

这也是 RAG 的魅力和危险:

- 魅力: 不局限于复制 passage span。
- 危险: 可能生成 passage 中没有的内容。

13. Table 2 和 Table 3: Generation and FEVER

Open MS-MARCO:

- RAG-Sequence 比 BART 提高约 2.6 Bleu points。
- Rouge-L 也提高约 2.6。
- 论文强调很多 MS-MARCO 问题并不能只靠 Wikipedia 回答, 所以 RAG 有时需要依赖 parametric knowledge。

Jeopardy question generation:

- RAG-Token 在 Jeopardy generation 上表现优于 RAG-Sequence。
- 原因可能是 Jeopardy 问题常需要组合多个事实。
- Figure 2 展示 token-level document posterior: 生成某些书名时, 对应 document posterior 高; 生成后续 token 时 posterior 变平, 说明参数记忆也参与补全。

Human evaluation:

- BART 更 factual 的情况只有 7.1%。
- RAG 更 factual 的情况是 42.7%。
- both factual 还有 17%。
- specificity 上 RAG 也明显更受偏好。

FEVER:

- RAG 在 3-way classification 上距离复杂 pipeline SOTA 约 4.3%。
- 这些 pipeline 通常使用强 retrieval supervision 和领域特定架构。
- RAG 没有使用 retrieved evidence 的中间监督。

Table 3 的 qualitative examples 展示 RAG 比 BART 更少 hallucination、更具体。

14. Ablations

Table 6 是理解 RAG 很重要的消融。

Frozen retriever:

- 冻结 retriever 时，多数任务变差。
- 说明 learned retrieval 对任务适配有帮助。

BM25 retriever:

- 用 BM25 替换 dense retriever，并用 BM25 scores 做 document logits。
- FEVER 上 BM25 表现很好，可能因为 FEVER claims 很 entity-centric，词面匹配有效。
- 其他任务尤其 open-domain QA 上，differentiable dense retrieval 更重要。

这给现代 RAG 工程的启发是：

BM25 并不低级。

Dense retriever 也不总赢。

任务类型决定检索器选择。

Index hot-swapping:

- 论文用 2016 Wikipedia index 和 2018 Wikipedia index 比较世界领导人问题。
- 替换 index 就能更新模型世界知识。
- 纯 parametric model 需要再训练才能更新类似事实。

Retrieved document count:

- 对 RAG-Sequence, NQ 性能随着 test-time retrieved documents 增多而单调提升。
- 对 RAG-Token, 性能在约 10 documents 达峰。
- 更多 docs 会增加 runtime，也可能增加噪声。

15. 数据形状

一个训练样本:

x:

question, claim, or answer entity

y:

answer text, generated question, or class label

一个 passage:

z:

passage text from Wikipedia
dense vector $d(z)$
optional metadata

Retriever output:

```
topK = [  
    (z_1, score_1, p_1),  
    (z_2, score_2, p_2),  
    ...  
    (z_K, score_K, p_K)  
]
```

Generator input for one document:

```
encoder_input = concatenate(x, z_k)
decoder_prefix = y_before_i
decoder_target = y_i
```

RAG-Sequence tensor intuition:

doc dimension:

K documents

token dimension:

N output tokens

for each doc k:

compute product over N token probabilities

then sum over K docs

RAG-Token tensor intuition:

for each token i:

compute K generator probabilities

weight by K retriever probabilities

sum over docs

then product over tokens

16. 本地代码映射

原论文机制:

learning/rag-essential/src/rag_original_minimal.py

对应:

- rag_sequence_prob: RAG-Sequence 边缘化。
- rag_token_prob: RAG-Token token-level 边缘化。
- dpr_inner_product: DPR 的 dot-product scoring。
- top_k_docs: MIPS top-k 的玩具版。
- hot_swap_answer: index hot-swapping 的玩具版。

现代 RAG 工程变体:

learning/rag-essential/src/naive_rag.py

learning/rag-essential/src/bm25_minimal.py

learning/rag-essential/src/hybrid.py

learning/rag-essential/src/reranker_mock.py

learning/rag-essential/src/colbert_minimal.py

learning/rag-essential/src/hyde_demo.py

learning/rag-essential/src/rag_fusion.py

learning/rag-essential/src/graph_rag.py

learning/rag-essential/src/hipporag.py

learning/rag-essential/src/self_rag.py

```
learning/rag-essential/src/ragas_metrics.py
learning/rag-essential/src/capstone_rag_compare.py
```

读法建议:

- 先读 `rag_original_minimal.py`, 理解论文概率模型。
- 再读 `naive_rag.py`, 理解现代 prompt-stuffing RAG。
- 再读 `hybrid.py`, 理解 BM25 + dense + RRF。
- 再读 `reranker_mock.py` 和 `colbert_minimal.py`, 理解 reranking。
- 最后读 GraphRAG、HippoRAG、Self-RAG、RAGAS, 理解后来工程如何补 RAG 缺点。

17. 最小代码片段

```
from rag_original_minimal import rag_sequence_prob, rag_token_prob
```

```
doc_probs = {"d1": 0.7, "d2": 0.3}
token_probs_by_doc = {
    "d1": [0.9, 0.8],
    "d2": [0.1, 0.95],
}
```

```
seq = rag_sequence_prob(doc_probs, token_probs_by_doc)
tok = rag_token_prob(doc_probs, token_probs_by_doc)
```

```
print(round(seq, 4))
print(round(tok, 4))
```

预期:

```
0.5325
0.5577
```

这个小例子展示:

- RAG-Sequence 要一个 document 解释整段输出。
- RAG-Token 可以让不同 token 从不同 document 获得支持。

Hot swap 片段:

```
from rag_original_minimal import hot_swap_answer

old_index = {"capital of exampleland": "Oldtown"}
new_index = {"capital of exampleland": "Newcity"}

query = "What is the capital of Exampleland?"

print(hot_swap_answer(query, old_index))
print(hot_swap_answer(query, new_index))
```

它对应论文的 index hot-swapping: 不改 generator 参数, 只替换 non-parametric memory。

18. 30 到 60 分钟本地实验

步骤 1: 跑环境和测试。

```
python learning\rag-essential\environment\verify_env.py
python learning\rag-essential\src\tests\test_rag.py
```

步骤 2: 单独跑原始 RAG 玩具模块。

```
python learning\rag-essential\src\rag_original_minimal.py
```

步骤 3: 修改 token_probs_by_doc。

建议实验:

- 让 d1 两个 token 都很强。
- 观察 RAG-Sequence 是否更接近或超过 RAG-Token。
- 让 d1 支持 token 1, d2 支持 token 2。
- 观察 RAG-Token 的优势。

步骤 4: 跑现代 RAG 对比。

```
python learning\rag-essential\src\capstone_rag_compare.py
```

观察:

- naive dense、BM25、hybrid、rerank、GraphRAG、HyDE 等策略在 toy metrics 上差异。
- faithfulness、answer relevancy、context precision、context recall 的含义。

步骤 5: 写 5 句话。

模板:

原始 RAG 的关键不是简单把文档塞进 prompt,
而是把 retrieved document 当 latent variable。
RAG-Sequence 在整段答案层面对文档求和。
RAG-Token 在每个 token 层面对文档求和。
现代工程 RAG 常常简化成 retrieve-then-generate,
但仍继承了 non-parametric memory 可更新和可检查的思想。

19. 证据链总结

证据一: Open-domain QA。

- RAG 在 NQ、WQ、CT 上达到 strong 或 SOTA 结果。
- 证明 hybrid memory 对知识密集 QA 有效。
- RAG 可以不使用 extractive reader 直接生成答案。

证据二: Generation tasks。

- MS-MARCO 上 RAG-Sequence 优于 BART。
- Jeopardy generation 上 RAG-Token 更适合组合多个文档。
- Human evaluation 认为 RAG 更 factual 和 specific。

证据三: FEVER。

- RAG 在无 evidence supervision 的情况下接近复杂 pipeline。
- 说明它可用于分类任务, 只要把 label 当作 output sequence。

证据四: Retrieval ablation。

- learned retrieval 多数任务优于 frozen retrieval。
- BM25 在 FEVER 上有优势。

- 说明检索器选择和任务形状有关。

证据五: Index hot-swapping。

- 替换 non-parametric memory 可以更新模型事实。
- 这是 pure parametric model 难做到的。

20. 局限性

第一, retrieval 失败仍会导致错误。

RAG 只是在 top K 上边缘化。如果 top K 都没有好证据, generator 仍可能 hallucinate 或依赖参数记忆。

第二, top K approximation 不是完整检索空间。

真实 Wikipedia 有海量 passages。RAG 只看 top K, 所以 retriever quality 决定上限。

第三, provenance 不等于忠实。

模型检索到某 passage, 不代表生成答案真的只依据该 passage。它可能同时使用 parametric memory。

第四, index 固定有利于工程, 但限制联合优化。

固定 document encoder 避免重建 index, 但也意味着 document representation 不能随下游任务一起学习。

第五, 成本和延迟上升。

RAG 增加 retrieval、top K document encoding、multiple forward passes 和 decoding 复杂度。

第六, 知识库选择决定边界。

论文主要使用 Wikipedia index。对于私有数据、实时数据、多模态数据或长尾领域, 需要重新构造 memory。

21. 现代意义

RAG 已经成为 LLM 应用的基础范式之一, 但现代工程常常使用更简单的流程:

```
chunk documents
embed chunks
retrieve top K
stuff context into prompt
generate answer
```

这和原始 RAG 的关系是:

- 继承了 non-parametric memory 的思想。
- 继承了 retrieved evidence 改善 factuality 的目标。
- 继承了 index hot-swapping 的可更新性。
- 但经常不再做 latent document marginalization。
- 也经常不端到端训练 retriever 和 generator。

后来发展补上了很多工程层。

Hybrid retrieval:

- BM25 + dense retrieval。

- 避免只靠语义向量或只靠词面匹配。

Reranking:

- cross-encoder 或 late interaction 重新排序。
- 提高 context precision。

Query rewriting:

- Multi-query、RAG-Fusion、HyDE。
- 改善 query 表达。

GraphRAG and HippoRAG:

- 用实体图、community summary、PageRank 处理 multi-hop。

RAGAS:

- 用 faithfulness、answer relevancy、context precision、context recall 评估 RAG。

Self-RAG:

- 让模型判断是否需要检索，检索是否有用，答案是否支持。

这些都是原始 RAG 思想的工程扩展。

22. 常见误解

误解一: RAG 就是向量检索。

不对。向量检索只是 non-parametric memory access。RAG 的完整问题还包括生成、证据利用、边缘化、训练、解码和评测。

误解二: 检索到正确文档就不会 hallucinate。

不对。generator 可能忽略证据、误读证据、混合参数记忆，或者过度补全。

误解三: Dense retriever 一定比 BM25 好。

不对。论文里 FEVER 上 BM25 很强。现代工程里 hybrid retrieval 也常常优于单一检索器。

误解四: RAG 能解决所有知识更新。

不对。RAG 能更新 index，但如果生成器不会正确使用新证据，或者 query 找不到新证据，仍会失败。

误解五: 原始 RAG 等于 LangChain prompt stuffing。

不对。prompt stuffing 是现代简化工程。原始 RAG 是 retrieval latent variable + marginal likelihood 的 probabilistic model。

23. 用 AI agent 正确学习这篇

你可以让学习 agent 这样考你。

我正在学习 Retrieval-Augmented Generation 论文。

请不要只说 RAG 是检索增强生成。

请按以下顺序考我:

1. parametric memory 和 non-parametric memory 分别是什么?
2. Figure 1 中 retriever 和 generator 怎么连接?

3. 为什么 document z 被当作 latent variable?
4. RAG-Sequence 和 RAG-Token 的公式差别是什么?
5. DPR 的 query encoder 和 document encoder 怎么打分?
6. 为什么训练时固定 document encoder 和 index?
7. RAG-Sequence decoding 为什么比 RAG-Token 麻烦?
8. Table 1 对 open-domain QA 证明了什么?
9. Jeopardy generation 为什么适合 RAG-Token?
10. Table 6 的 BM25/Frozen retriever ablation 说明什么?
11. index hot-swapping 为什么重要?
12. 本仓库 rag_original_minimal.py 如何复现边缘化?

每次只问一个问题。

我回答后，请要求我画公式或跑本地代码。

最后让我闭卷讲出 RAG-Sequence 和 RAG-Token 的区别。

24. 闭卷掌握检查

1. 为什么纯 parametric LM 在知识密集任务上有局限?
2. non-parametric memory 在 RAG 中具体是什么?
3. Figure 1 的 query encoder、document index、generator 各做什么?
4. DPR 的 score 如何计算?
5. MIPS 在这里解决什么问题?
6. RAG 为什么把 document z 当 latent variable?
7. RAG-Sequence 的边缘化公式是什么?
8. RAG-Token 的边缘化公式是什么?
9. 哪种模型更适合一个答案需要多个文档支持的场景?
10. 为什么 RAG-Sequence decoding 更麻烦?
11. 训练时哪些参数更新，哪些固定?
12. Table 1 中 RAG 对 open-domain QA 的主要结论是什么?
13. RAG 为什么能在 answer 不直接出现在 retrieved document 时仍答对一部分?
14. Jeopardy generation 中 Figure 2 的 document posterior 说明了什么?
15. FEVER 上 RAG 的意义是什么?
16. Table 6 中 BM25 在 FEVER 上强说明了什么?
17. index hot-swapping 如何更新模型知识?
18. 为什么 provenance 不等于 faithful generation?
19. 本地 rag_original_minimal.py 中哪个函数对应 RAG-Sequence?
20. 现代 prompt-stuffing RAG 和原始 RAG 最大区别是什么?

25. 最小复述模板

闭卷时可以这样讲:

RAG 论文提出把预训练 seq2seq 模型的参数记忆和 Wikipedia dense index 这样的非参数记忆结合起来。给定输入 x , DPR retriever 用 query encoder 和 document encoder 做内积检索 top K passages z , BART generator 在 x 和 z 条件下生成 y 。核心数学是把 z 当作 latent variable。RAG-Sequence 假设同一个 document 解释整个输出, 所以先对每个 document 计算整段生成概率, 再按 retriever probability 求和。RAG-Token 则在每个 token 位置对 documents 求和, 因此更适合组合多个文档。训练目标是 negative marginal log likelihood, 更新 query encoder 和 generator, 固定 document encoder 和 index。实验显示 RAG 在 open-domain QA 上强, 在 MS-MARCO、Jeopardy 和 FEVER 上也有效, 并且可以通过替换 index 更新知识。它的局限是检索错误、top K 近似、生成不忠实、成本和延迟。现代 RAG 工程继承了非参数记忆思想, 但很多实现已简化为 retrieve-then-prompt。

能讲出这段, 并能跑 rag_original_minimal.py 解释 0.5325 和 0.5577 的区别, 这篇就是真的进脑子了。